

# Assignment 1 — Weeks 1 & 2

Database Management Systems & ER Modelling

|        |                       |          |                        |
|--------|-----------------------|----------|------------------------|
| Name   | Saksham Agarwal       | Roll No. | 250954                 |
| Branch | Aerospace Engineering | Email    | sakshamag25@iitk.ac.in |

## Part I — Short Answer Questions

### Q1. DBMS vs. Traditional File System

Before databases became mainstream, most applications stored their data in ordinary files managed by the operating system. Each program defined its own file format and wrote its own code to read and write it. This worked for small, single-user applications but broke down in more complex settings. If two departments needed the same customer data, each kept its own copy — and those copies regularly fell out of sync. Two users writing to the same file simultaneously could corrupt it entirely. And there was no built-in way to query the data; every question required custom parsing code.

A DBMS solves all of this by sitting as a layer between the application and the raw data. It provides a common query language (SQL), enforces integrity rules so invalid data can never get in, handles concurrent users safely through transactions, and controls who is allowed to see what. The application no longer needs to manage any of these concerns — the DBMS handles them centrally.

#### Concrete example — Hospital Patient Management:

Consider a hospital where the admissions desk, pharmacy, and billing department each maintain separate patient files. When a patient's address changes, someone must update three different files manually — miss one and the records become inconsistent. If two nurses update the same patient file simultaneously, one change is lost. With a DBMS, patient data lives in one place. Every department reads from the same source via foreign key relationships, so an address update is reflected everywhere instantly. Concurrent writes are managed by the transaction engine. A referential integrity constraint prevents a prescription from being written for a patient ID that does not exist. This is exactly the kind of scenario where a DBMS is not just better — it is the only sensible choice.

### Q2. Weak Entity vs. Strong Entity

A **strong entity** can stand on its own. It has enough attributes to uniquely identify each of its instances without relying on any other entity. A Person has a unique national ID; a Product has a unique product code. Strong entities are drawn as regular rectangles in an ER diagram.

A **weak entity** cannot identify itself alone. It depends on another entity — its **identifying (owner) entity** — for part of its identity. A classic example is a bank account's Transaction: two transactions from different accounts might share the same transaction number, so the number alone is not

unique globally. The transaction only makes sense in the context of a specific account. In an ER diagram, a weak entity is drawn with a **double rectangle**, and the relationship tying it to its owner uses a **double diamond**.

#### Primary key formation:

The weak entity has a **partial key** (shown with a dashed underline) — an attribute unique only among instances belonging to the same owner. The full primary key combines this partial key with the primary key of the identifying entity. For example, if Dependent is a weak entity owned by Employee, the partial key might be the dependent's name. Two employees could each have a dependent called 'Aisha', so the name alone is not globally unique. The composite key (employee\_id, dependent\_name) uniquely identifies a dependent. This connecting relationship is called the **identifying relationship**, and the weak entity always has total participation in it — a dependent cannot exist without an employee.

### Q3. Modelling Student Phone Numbers

Phone number should be modelled as a **multivalued attribute**. The problem statement itself provides the answer: a student *may have multiple* phone numbers. A simple attribute holds exactly one value per entity instance — using it here would restrict each student to a single number, contradicting the requirement. A composite attribute is also wrong here; composites are for attributes made up of independently meaningful sub-parts, like an address (street, city, PIN). A phone number has no such sub-structure that would need to be stored or queried separately.

In an ER diagram, a multivalued attribute is represented with a **double ellipse**. When mapping to a relational schema, it is extracted into a separate table — Student\_Phone(roll\_no, phone\_number) — where each row stores one number for one student, and the composite (roll\_no, phone\_number) serves as the primary key. This is the clean, normalised approach.

### Q4. Candidate Key vs. Super Key

A **super key** is any set of attributes that, taken together, can uniquely identify every row in a relation. There is no restriction on size — extra attributes can always be added and the set remains a valid super key, since more information only makes identification more specific.

A **candidate key** is a super key with the additional property of **minimality (irreducibility)**: removing any single attribute from it breaks uniqueness. Every relation must have at least one candidate key; we designate one as the primary key and enforce the rest as UNIQUE constraints.

#### Applied to: Student(roll\_no, email, name, branch)

Assuming roll\_no and email are each unique per student:

- **Candidate keys:** {roll\_no} and {email} — each is sufficient and minimal on its own.
- **Super keys that are not candidate keys (non-minimal examples):**
  - {roll\_no, email} — unique, but email is redundant since roll\_no alone already identifies the student.
  - {roll\_no, name} — unique because roll\_no is unique, but name is unnecessary.

– {roll\_no, email, name, branch} — the complete tuple; unique but carries three redundant attributes.

### Q5. Inserting a Row with a Non-Existent Student ID in Enrollment

#### Constraint violated: Referential Integrity Constraint (Foreign Key Constraint)

The Enrollment table contains a foreign key column `student_id` referencing the primary key of the Student table. This constraint requires that every value in the foreign key column must already exist as a primary key value in the referenced table. Inserting a `student_id` that has no matching row in Student directly violates this rule.

#### What it protects against:

Referential integrity prevents **orphaned records** — rows in Enrollment pointing to a student who does not exist. Without this safeguard, a JOIN between Enrollment and Student would silently discard those rows, making counts and reports inaccurate. You could end up with enrolment statistics that do not match the student count, or application errors when attempting to load details for a non-existent student.

#### What should happen to the insert:

The DBMS must **reject the insert and roll it back**, returning a foreign key violation error to the calling application. The row must not be written to the database. The correct resolution is to either create the student record in the Student table first, or to validate that the `student_id` exists before attempting the enrolment insert.

### Q6. Mapping a Many-to-Many (M:N) Relationship to a Relational Schema

#### Additional structure required:

A **junction table** (also called a bridge table or associative table) must be created. This new relation represents the M:N relationship itself as a proper structure in the schema, with its own rows and columns.

#### Primary key composition:

The junction table's primary key is a **composite key** formed from the primary keys of both participating entities. For a Student–Course relationship, the junction table Enrollment would have the primary key (`student_id`, `course_id`), ensuring the same student cannot be enrolled in the same course more than once.

#### Why a column in one entity table does not work:

In a 1:N relationship, a foreign key on the 'many' side is sufficient. In an M:N relationship, however, both sides can link to multiple entities on the other side. Adding a `course_id` column to the Student table would restrict each student to one course — a 1:N constraint, not M:N. Storing multiple course IDs in a single column as a delimited list violates First Normal Form and makes querying impractical. The junction table is the only correct solution: each row represents exactly one student–course pairing, and relationship-specific data such as a grade or enrolment date can be added as further columns.

## Part A — Entity and Relationship Identification

### Entities and their key attributes:

| Entity     | Primary Key   | Other Attributes                                                        |
|------------|---------------|-------------------------------------------------------------------------|
| Customer   | customer_id   | name, email, phone_numbers (multivalued), delivery_address (composite)  |
| Restaurant | restaurant_id | name, address, cuisine_type, contact_number, rating                     |
| MenuItem   | item_id       | item_name, description, price, category, is_available                   |
| Order      | order_id      | order_date, order_time, total_amount, delivery_status, delivery_address |

### Relationships, cardinalities, and justifications:

| Relationship          | Cardinality | Justification                                                                                   |
|-----------------------|-------------|-------------------------------------------------------------------------------------------------|
| Customer — Order      | 1 : N       | A customer can place many orders over time, but each order belongs to exactly one customer      |
| Restaurant — Order    | 1 : N       | A restaurant receives many orders, but each order is placed with exactly one restaurant         |
| Restaurant — MenuItem | 1 : N       | A restaurant offers many menu items, but each item belongs to one specific restaurant           |
| Order — MenuItem      | M : N       | A single order can contain multiple items, and any given item can appear across multiple orders |

## Part B — Attribute Classification

### Multivalued attribute — Customer phone numbers

A customer might register a personal mobile, a work number, and a separate number for delivery contact. Since the real-world situation permits more than one value but a simple attribute can hold only one, phone\_number qualifies as a **multivalued attribute**. It is shown with a double ellipse in the ER diagram. In the relational schema it becomes a separate table — Customer\_Phone(customer\_id, phone\_number) — so each number gets its own row.

### Composite attribute — Delivery address

A delivery address is composed of independently meaningful sub-parts: flat/house number, street, locality, city, state, and PIN code. Storing it as a single string prevents efficient querying — finding all orders delivered to a particular city would require fragile string matching. Modelling it as a **composite attribute** keeps the data structured and queryable, with each sub-component represented as a child ellipse in the ER diagram.

## Part C — Relational Schema for the Junction Table

The M:N relationship between Order and MenuItem requires a junction table. The complete schema is:

```
OrderItem(order_id, item_id, quantity, unit_price)
```

| Column   | Data Type | Role      | Notes                                |
|----------|-----------|-----------|--------------------------------------|
| order_id | INT       | PK + FK   | References Order(order_id)           |
| item_id  | INT       | PK + FK   | References MenuItem(item_id)         |
| quantity | INT       | Attribute | Units ordered; NOT NULL, must be > 0 |

|            |              |           |                                                          |
|------------|--------------|-----------|----------------------------------------------------------|
| unit_price | DECIMAL(8,2) | Attribute | Price at time of ordering — menu prices may change later |
|------------|--------------|-----------|----------------------------------------------------------|

The composite primary key (order\_id, item\_id) ensures each item appears at most once per order — multiple units of the same dish are handled by setting quantity accordingly, not by adding duplicate rows. Both columns also serve as foreign keys maintaining referential integrity with their parent tables. Storing unit\_price as a snapshot is a deliberate design decision: menu prices change over time, and always looking up the current price would produce incorrect totals for past orders.

Part III — ER Diagram Design

Question 1: Hospital ER Diagram

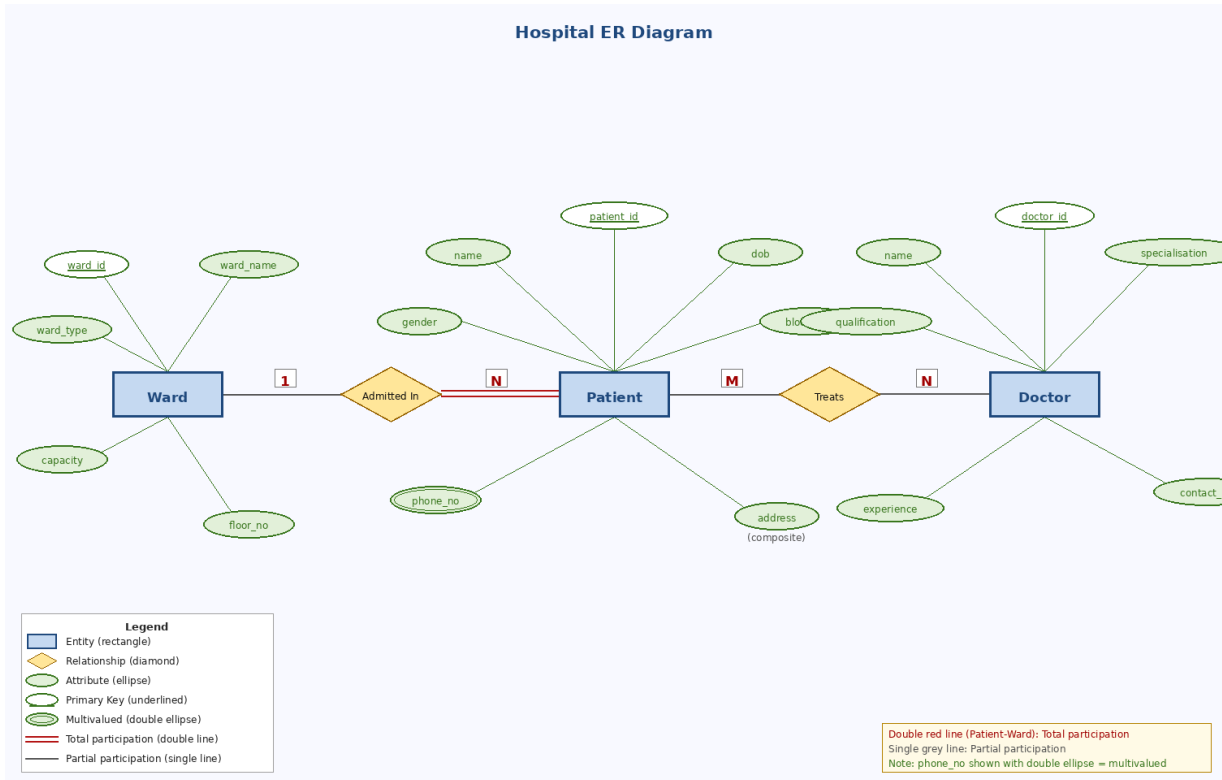


Figure 1: Hospital ER Diagram — Patient, Doctor, Ward | Double red lines = total participation (Patient–Ward) | Double ellipse = multivalued attribute (phone\_no)

Entities, primary keys, and attributes:

| Entity  | Primary Key | Other Attributes                                                          |
|---------|-------------|---------------------------------------------------------------------------|
| Patient | patient_id  | name, dob, gender, blood_grp, address (composite), phone_no (multivalued) |
| Doctor  | doctor_id   | name, specialisation, qualification, contact_no, experience               |
| Ward    | ward_id     | ward_name, ward_type (ICU/General/Surgical), capacity, floor_no           |

Relationships, cardinalities, and participation constraints:

| Relationship               | Cardinality | Patient / Doctor side                                                   | Ward / Doctor side                                                      |
|----------------------------|-------------|-------------------------------------------------------------------------|-------------------------------------------------------------------------|
| Ward–Patient (Admitted In) | 1 : N       | Total — every admitted patient must be assigned to a ward (double line) | Partial — a patient may be temporarily in no patient ward (single line) |
| Doctor–Patient (Treats)    | M : N       | Partial — a patient may not yet have a doctor assigned (single line)    | Partial — a doctor may not yet have any patients (single line)          |

Question 2: Design Justification

1. Why Doctor–Patient is M:N, the associative entity it creates, and its composite key

During a typical hospital stay, a patient may be seen by more than one doctor — a general physician who first assesses them, a cardiologist who reviews the ECG, and a physiotherapist managing recovery. That is one patient linked to multiple doctors. On the other side, each doctor

simultaneously manages a caseload of multiple patients. Neither side is limited to a single partner, which is the definition of a many-to-many relationship.

Since M:N relationships cannot be represented directly in a relational schema, this one produces an **associative entity** called **Treatment**. Each row in the Treatment table records one specific doctor–patient interaction. Its **composite primary key** is (doctor\_id, patient\_id). If the same doctor treats the same patient across multiple visits, a third component — treatment\_date or a surrogate treatment\_id — is added to keep rows distinct. Additional attributes of Treatment might include diagnosis\_notes, treatment\_type, and prescription\_details.

## 2. Participation constraints for two relationships

**Ward–Patient (Admitted In) — Total on the Patient side, Partial on the Ward side:** This database tracks patients who have been admitted to the hospital. Once a patient is admitted, they must be placed in a ward; there is no concept of an admitted patient without a ward assignment in this system. This is total participation from the Patient side, shown with a double line in the ER diagram. The Ward side carries partial participation: a ward is a physical unit that exists regardless of whether it currently has any patients. It might be fully occupied, partially filled, or empty after a round of discharges — it remains a valid entity in all cases.

**Doctor–Patient (Treats) — Partial on both sides:** A doctor who has just joined the hospital and has not yet been assigned to any patient is still a valid Doctor record. Similarly, a patient's record may be created during initial registration before any doctor has been allocated. Because it is entirely possible for either entity to exist without participating in the Treats relationship, both sides carry partial participation, shown as single lines on both connections in the diagram.

## 3. Attribute design decision — phone\_no modelled as a multivalued attribute for Patient

The Patient entity includes contact phone numbers, modelled as a **multivalued attribute** (double ellipse). A patient may want to register their own mobile, a family member's number for emergency contact, and a home landline. Forcing phone\_no to be single-valued would either lose this information or require storing a comma-separated list in one field — which violates First Normal Form and makes individual numbers impossible to index or search. The multivalued attribute design maps cleanly to a separate table in the relational schema: Patient\_Phone(patient\_id, phone\_number), where each number has its own row and the pair forms the primary key. This keeps the main Patient table clean, normalised, and easy to query.